# 适合课程小车的软件架构研究报告

| | |
|---|---|
| 研究对象 | TC264D 裸机小车控制软件 |
| 推荐架构 | 分层架构、周期任务调度与事件驱动相结合的轻量级混合架构 |
| 当前基础 | CPU0 主循环、CCU60 PIT、按键模块、电机模块、编码器和双轮速度 PID |
| 研究范围 | 架构介绍、优缺点、课程小车实施方案、目录结构和核心代码结构 |

## 1 研究背景与目标

课程小车不是单一的“让电机转起来”程序。随着功能增加，软件需要同时处理按键、编码器、PID、循迹传感器、姿态传感器、避障和通信。如果所有代码都直接堆叠在一个主循环或一个中断函数中，程序虽然可以快速运行，但模块之间会形成强耦合，调试和扩展会越来越困难。

本报告研究一种适合当前 TC264 小车的轻量级软件架构。研究目标如下：

1. 说明推荐架构的组成、职责边界和数据流；
2. 分析该架构对课程小车的优点和限制；
3. 将架构落实到当前裸机工程的启动、主循环和 PIT 周期任务；
4. 给出可直接用于工程整理的目录结构、模块接口和核心代码组织；
5. 为循迹、速度 PID、姿态控制和避障等后续功能预留清晰的扩展边界。

## 2 推荐架构概述

### 2.1 架构定义

本文推荐的架构是：

**基于裸机的分层、周期调度和事件驱动混合架构**。

它不是引入操作系统，而是在现有 CPU0 主循环和 PIT 中断基础上，借鉴实时系统的任务组织思想：

- **分层：**底层负责芯片和外设，中间层负责可复用功能模块，应用层负责小车行为决策；
- **周期调度：**把必须按固定时间执行的采样和控制任务按周期组织；
- **事件驱动：**按键、串口命令、故障和状态变化通过事件进入应用层，而不是由驱动层直接修改业务状态；
- **数据单向流动：**传感器数据向上汇聚，控制命令向下传递，减少模块相互调用。

### 2.2 总体结构

| 应用层：模式管理、运动目标、故障策略、任务编排 |
| --- |
| ↓ 目标、命令、事件　↑ 状态、测量值 |
| 控制与功能模块层：速度 PID、循迹、姿态、避障、按键、通信 |
| ↓ 模块接口 |
| 设备抽象层：电机、编码器、传感器、显示、UART |
| ↓ 驱动调用 |
| TC264 底层：GPIO、PWM、GPT12、PIT、UART、SPI |

### 2.3 运行流程

启动复位 → 时钟和底层驱动 → 模块初始化 → 启动周期定时器
主循环：处理事件、更新模式、发布控制目标、处理低频通信
PIT：执行短小的周期采样、计时、置任务标志和必要的快速控制
模块：读取自己的输入，更新内部状态，通过接口提供测量值或输出目标

## 3 架构组成与职责

### 3.1 底层驱动层

底层驱动层直接面对 TC264 和现有库接口，负责 GPIO、PWM、GPT12 编码器、PIT、UART 和 SPI 等硬件资源。该层的原则是“提供稳定的硬件能力，不包含小车业务规则”。

例如，PWM 驱动只负责设置占空比，不能在驱动内部决定“循迹时左轮应该减速”；编码器驱动只负责提供计数值，不能在驱动内部修改 PID 积分。

### 3.2 设备抽象层

设备抽象层把芯片外设包装成小车可理解的设备接口：

- 电机设备：接收左右轮有符号输出，处理 A/B 或方向/PWM 的硬件极性；
- 编码器设备：提供左右轮增量、累计位置和方向；
- 传感器设备：提供循迹、姿态和距离的原始或初步处理数据；
- 显示和通信设备：提供状态显示和参数收发。

该层隔离“端口变化”。例如，本次开发中电机接口从最初假设的“方向 +PWM”修正为 A/B 双输入 H 桥。如果应用层只依赖 `motor_set()`，端口映射变化就不会扩散到 PID 和应用代码。

### 3.3 功能控制模块层

功能模块层是架构的核心复用层，包含：

- 按键模块：输入采样、消抖、短按/长按事件；
- PID 模块：通用误差计算、积分限幅、输出限幅；
- 速度控制模块：编码器采样、左右轮 PID 实例和目标管理；

- 循迹模块：传感器误差计算和路径偏差输出；
- 姿态模块：陀螺仪/加速度计数据处理和角度控制；
- 避障模块：距离判断、障碍事件和安全停车请求；
- 通信模块：参数设置、状态读取和调试命令。

功能模块应该有清晰输入和输出，尽量不直接调用其他业务模块的内部变量。

### 3.4 应用层

应用层负责回答“当前小车要做什么”，而不是回答“某个 GPIO 怎么配置”。应用层可以维护运行模式：

- 手动模式：按键或通信设定左右轮目标；
- 速度控制模式：目标速度进入左右轮 PID；
- 循迹模式：循迹误差转换为左右轮差速目标；
- 避障模式：故障或障碍事件触发减速、停车或转向；
- 故障安全模式：传感器异常、通信超时或控制量非法时停止电机。

## 4 周期任务与事件模型

### 4.1 周期任务原则

不同任务有不同的实时性要求。架构不把所有逻辑都放在同一个周期中，而是按“采样和控制的时间约束”分层：

表 1: 建议的周期任务类别

| 类别 | 典型内容 | 原则 |
| --- | --- | --- |
| 快速基础任务 | 电机输出刷新、时间基准、输入采样 | 代码短、执行时间稳定，不做字符串处理 |
| 控制任务 | 编码器测速、速度 PID、姿态控制 | 周期稳定，使用上一次已采样数据，避免阻塞 |
| 中速任务 | 循迹计算、状态机、传感器融合 | 可由任务标志触发，不能阻塞快速控制 |
| 低速任务 | UART 状态、参数保存、显示刷新 | 放在主循环或低频调度，不进入高优先级 ISR |

周期数值不应脱离传感器和执行器特性随意确定。当前工程已经验证 1 ms PIT 基础节拍和 10 ms 速度 PID 周期；后续循迹、姿态和通信任务应根据实际采样率再分配。

### 4.2 中断职责

当前工程的 `user/isr.c` 统一放置中断函数。推荐的 ISR 职责是：

1. 清除中断标志；
2. 完成非常短的采样或计时；
3. 更新必要的硬件输出；

4. 递增软件计时器并置任务标志；
5. 退出中断。

复杂的字符串解析、模式切换、参数检查和故障策略不应直接放在 ISR 中。这样可以降低中断最长执行时间，避免低优先级任务长期得不到运行机会。

### 4.3 事件模型

事件是“已经发生的事实”，例如：

```
KEY_EVENT_SHORT
KEY_EVENT_LONG
OBSTACLE_DETECTED
UART_COMMAND_READY
CONTROL_TIMEOUT
SENSOR_INVALID
```

事件生产者只负责产生事件，应用层决定如何处理事件。按键模块不直接调用电机模块；它只报告 KEY3 或 KEY4 的事件，应用层再把事件转换为目标速度或停车命令。

## 5 结合本课程小车的实施方案

### 5.1 现有工程映射

当前工程可以按如下方式归类：

表 2: 当前工程到推荐架构的映射

| 现有对象 | 架构层 | 职责 |
|---|---|---|
| zf_driver_* | 底层驱动层 | 调用 TC264 外设能力 |
| motor.c/.h | 设备抽象层 | 隔离 A/B 电机端口和输出极性 |
| key_module.c/.h | 功能模块层 | 按键消抖和事件产生 |
| pid.c/.h | 控制算法层 | 通用 PID 计算 |
| speed_control.c/.h | 控制算法层 | 编码器反馈和左右轮闭环 |
| app.c/.h | 应用层 | 目标速度、方向、模式和用户交互 |
| user/isr.c | 调度入口 | PIT 节拍和周期任务触发 |

### 5.2 启动和主循环

启动流程应保持简单、确定：

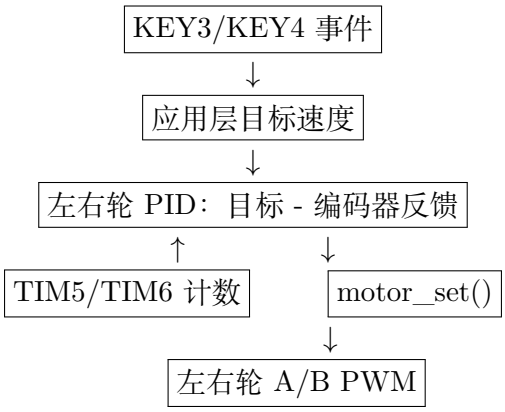
```
int core0_main(void)
{
    clock_init();
    debug_init();
    board_init();
```

```
    motor_init();
    encoder_init();
    control_init();
    input_init();
    scheduler_init();

    while(TRUE)
    {
        scheduler_run_ready_tasks();
        app_process_events();
        communication_task();
    }
}
```

这里的 board_init() 可以是对现有初始化函数的组织性封装；在不增加复杂度的情况下，也可以保留当前 app_init()，但应逐步将初始化职责拆到相应模块。

## 5.3 当前 PID 控制数据流




当前速度环使用两个独立 PID 实例。每次控制更新读取 TIM5 和 TIM6 的增量计数，分别计算左、右轮输出，再将输出交给电机模块。电机模块负责输出极性、A/B 通道映射和缓启动，PID 模块不直接操作 GPIO。

## 5.4 未来功能扩展

表 3: 面向后续功能的接口预留

| 功能 | 新增模块 | 与现有架构的连接 |
|---|---|---|
| 循迹 | line_sensor、line_control | 输出左右轮差速目标，不修改电机底层 |
| 姿态/角度 | imu、attitude_control | 输出航向或角度目标，复用 PID 算法 |
| 避障 | distance_sensor、safety_manager | 产生障碍事件和限速/停车请求 |

| 功能 | 新增模块 | 与现有架构的连接 |
| --- | --- | --- |
| 通信 | protocol、parameter_service | 修改目标和参数，读取状态，不直接操作 PWM |

# 6 架构的优点

## 6.1 模块职责清晰

电机端口变化只影响电机模块，编码器计数方式变化只影响编码器或速度控制模块，应用层不需要了解寄存器和端口细节。这一点在本课程实际调试中非常重要：电机最初被误判为“方向 +PWM”，后来根据板上 A/GND/B 标识改成 A/B 控制，最终只需调整电机模块。

## 6.2 适合实时控制

PID 和编码器采样具有明确周期，可以从主循环中的非确定性处理里分离出来。事件处理、显示和通信不应阻塞控制任务，从而使电机控制周期更稳定。

## 6.3 便于测试和定位

每层都可以单独测试：

- 不接电机时测试 PID 数学计算；
- 模拟编码器增量测试速度控制；
- 单独测试电机 A/B 映射；
- 最后进行按键、控制器和执行器的集成测试。

## 6.4 便于扩展

循迹和姿态控制可以复用 PID 算法；避障可以通过安全管理器限制目标速度；通信可以成为参数服务，而不必修改电机驱动和中断底层。

# 7 架构的缺点与风险

## 7.1 需要明确数据所有权

裸机架构没有操作系统自动管理任务和资源。共享变量如果同时在 ISR 和主循环中访问，必须明确其类型、读写方向和临界区策略，否则可能出现数据撕裂或状态覆盖。

## 7.2 调度能力有限

当任务数量增加时，基于软件计数器的调度器会逐渐变得复杂。如果某个任务执行时间过长，可能影响其他任务。必须避免在快速周期中进行阻塞通信、复杂格式化和长循环。

### 7.3 故障隔离较弱

裸机程序中一个模块越界、死循环或错误关闭中断，可能影响整个小车。架构需要增加参数检查、任务超时、传感器有效性判断和统一安全停车出口。

### 7.4 调参和状态观测成本较高

没有 RTOS 和标准诊断框架时，必须自行设计状态快照、串口协议或 TFT 调试页面。当前 UART 回传未稳定使用，说明架构不仅要设计控制路径，也要设计不依赖单一通信链路的观测手段。

## 8 目录结构与核心代码结构

### 8.1 建议目录结构

```
E01_gpio_demo/
|-- code/
| |-- app.c / app.h 应用状态机和目标管理
| |-- scheduler.c / scheduler.h 轻量任务调度
| |-- pid.c / pid.h 通用PID
| |-- speed_control.c / .h 编码器速度环
| |-- motion_interface.h 运动命令和反馈接口
| |-- motor.c / motor.h 电机设备抽象
| |-- encoder.c / encoder.h 编码器设备抽象
| |-- key_module.c / .h 按键事件
| |-- line_sensor.c / .h 循迹传感器
| |-- imu.c / .h 姿态传感器
| |-- safety_manager.c / .h 故障和避障
| `-- protocol.c / .h 通信协议
|-- user/
| |-- cpu0_main.c 启动入口
| |-- isr.c 所有中断函数
| `-- isr_config.h
|-- libraries/ 只调用，不修改
|-- 实验报告/
| |-- PID速度环报告/
| `-- 小车软件架构研究报告/
`-- AI对话日志.md
```

### 8.2 核心接口结构

```
typedef struct
{
    int16 left_target;
    int16 right_target;
    uint8 enable;
    uint8 safety_stop;
} motion_command_t;
```

```
typedef struct
{
    int16 left_speed;
    int16 right_speed;
    uint8 encoder_valid;
    uint32 timestamp_ms;
} motion_feedback_t;

void app_process_events(void);
void speed_control_set_target(int16 left, int16 right);
void speed_control_update(void);
void motor_set(int16 left, int16 right);
void safety_manager_request_stop(void);
```

本阶段已将运动接口实际同步到工程。应用层通过 speed_control_set_command() 提交结构化运动命令，速度控制层通过 speed_control_get_feedback() 提供反馈快照。原有的 speed_control_set_target() 和分项读取接口暂时保留为兼容接口，因此接口整理不会改变已经验证的 PID、电机映射和按键行为。

```
motion_command_t command;
motion_feedback_t feedback;

command.left_target = target;
command.right_target = target;
command.enable = 1U;
command.safety_stop = 0U;
speed_control_set_command(&command);
speed_control_get_feedback(&feedback);
```

## 8.3 任务标志核心代码

```
volatile uint8 task_flags;

void cc60_pit_ch0_isr(void)
{
    pit_clear_flag(CCU60_CH0);
    key_module_scan();
    motor_update();

    task_flags |= TASK_1MS;
    if((system_tick_ms % CONTROL_PERIOD_MS) == 0U)
    {
        task_flags |= TASK_CONTROL;
    }
}

void scheduler_run_ready_tasks(void)
{
    if(task_flags & TASK_CONTROL)
```

```
    {
        task_flags &= (uint8)~TASK_CONTROL;
        speed_control_update();
    }
    if(task_flags & TASK_APP)
    {
        task_flags &= (uint8)~TASK_APP;
        app_process_events();
    }
}
```

示例强调的是职责结构，而不是要求立即复制全部代码。实际工程中还应根据编译器对并发访问的要求使用合适的临界区、原子类型或事件队列。

# 9 实施路线与验收标准

表 4: 推荐实施路线

| 阶段 | 内容 | 验收标准 | 产物 |
|---|---|---|---|
| 1 | 固定端口和模块接口 | 电机、编码器、按键接口清晰 | 接口表 |
| 2 | 保持现有电机和 PID | 开环和闭环行为不改变 | 回归测试记录 |
| 3 | 引入轻量调度器 | 控制任务与低频任务分离 | 调度模块 |
| 4 | 增加状态和故障管理 | 异常输入可触发安全停止 | 安全模块 |
| 5 | 接入循迹和姿态 | 新模块不修改底层驱动接口 | 扩展模块 |
| 6 | 完成参数和通信服务 | 参数边界、超时和恢复行为明确 | 协议和测试表 |

## 9.1 当前阶段实施结果

接口整理的第一阶段已经同步写入工程：

- 新增 `motion_interface.h`，定义 `motion_command_t` 和 `motion_feedback_t`；
- `speed_control` 新增结构化命令输入和反馈输出接口；
- `app.c` 改为构造运动命令，不直接依赖速度控制模块的内部标量；
- 保留旧接口作为兼容层，当前电机映射、PID 参数和按键行为不变；
- 使用工程脚本重新构建，结果为 `0 errors, 0 warnings`。

# 10 结论

对于本课程 TC264 小车，不引入 RTOS 的轻量级混合架构更合适。它保留当前工程已经验证的裸机启动方式和 PIT 节拍，同时通过分层、周期任务和事件驱动建立清晰的软件边界。

该架构的关键不是增加大量框架代码，而是明确三条规则：

1. 硬件端口和极性只由设备抽象层管理；

2. 固定周期的采样和控制按照实时约束调度；
3. 按键、通信和故障等事实通过事件进入应用层，由应用层决定行为。

因此，当前双轮速度 PID 可以作为架构的第一个闭环控制模块；未来循迹、姿态、避障和通信功能可以在不破坏已有电机和控制接口的前提下逐步加入。

# A 本次需求确认对话原文

**说明：**以下保留本次架构研究需求确认阶段的用户消息和助手答复原文。

**用户：**

现在另开一个报告，我要完成这个任务：研究一下适合本课程小车的软件架构，其内容需要包含架构的介绍，架构的优缺点，结合本课程小车，该架构应该如何实施，目录结构和核心代码结构。要求报告结构合理，逻辑完整。同样是询问我需求，让我逐一确认。

**助手：**

这次我会新建一份独立的“软件架构研究报告”，不修改刚完成的 PID 实验报告；我会按“每轮确认一组需求”的方式推进，全部确认后再写报告。先确认研究范围和报告形式。

**助手：**

第一步请确认架构研究的主线和交付格式。结合当前 TC264 小车已有按键、PIT 中断、电机、编码器和 PID 模块，我建议以“分层架构 + 周期任务调度 + 事件驱动”的混合架构为主。

**用户：**

architecture_focus=hybrid_recommended，comparison_scope=no_comparison，report_format=latex_pdf，report_location=new_folder。

**助手：**

已确认：新报告采用“分层 + 周期任务 + 事件驱动”的混合架构，不展开其他架构对比，使用 LaTeX+PDF，并放入新的独立文件夹。下一步确认课程要求、篇幅和技术深度。

**助手：**

第二步请确认报告面向对象、篇幅和技术深度。为了让内容既像课程研究报告又能落到当前工程，我建议包含架构图、任务时序、目录树、核心伪代码/接口示例和当前小车的落地映射。

**用户：**

audience=course_teacher,length_level=standard,technical_depth=implementation_level,include_diagrams=true。

**助手：**

已确认报告面向课程教师，篇幅标准约 10–15 页，技术深度落到当前 C 文件、ISR、PID 周期和数据流，并加入图示。下一步确认架构实施边界：是基于当前无 RTOS 裸机工程演进，还是可以设计成 RTOS 架构。

**用户：**

什么是 RTOS?

**助手：**

RTOS 是实时操作系统。当前工程不需要引入 RTOS，建议采用基于裸机的轻量级实时混合架构，保留主循环和 PIT，但按周期任务和事件组织功能。

**用户：**

不需要。

**助手：**

明白，报告不引入 RTOS，也不展开 RTOS 设计；架构限定为当前 TC264 裸机环境下的轻量级混合架构。现在确认最后一组内容：任务周期、未来功能范围，以及是否保留本次架构报告的对话原文。

**用户：**

task_periods=flexible_periods，future_features=navigation，preserve_dialogue=true。